

Декомпозиция подхода «Все порты открыты»: Анализ Deception-систем со стороны Red Team

 redsec.by/article/dekompoziciya-podhoda-vse-porty-otkryty-analiz-deception-sistem-so-storony-red-team

Red Teams | 8 апреля 2026



В современной корпоративной защите под термином **«все порты открыты»** практически никогда не подразумевается реальный запуск десятков тысяч продуктивных сервисов на одном хосте. В 2026 году этот подход стал синонимом развертывания *децептивных схем (Active Defense / Deception)*. Это архитектурный трюк, при котором каждый TCP-порт логически отвечает атакующему как open. Реализуется это через перенаправление трафика на один или несколько демонов-ловушек (Portspooft, самописные эмуляторы, honeypot-фреймворки).

Цель Blue Team очевидна: превратить быстрый, автоматизированный и информативный порт-скан в шумный, бесконечно долгий и абсолютно бесполезный процесс. Истинные сервисы скрываются в море фейковых ответов, а каждое входящее соединение генерирует высококачественную телеметрию для SOC. Однако, с точки зрения профессионального инженера **Red Team**, этот прием не является непреодолимым барьером. Это «смоляная яма» (tarbit), которая заставляет менять тактику, но при наличии глубокого понимания протоколов (protocol-aware) и методов контр-децепции, реальную инфраструктуру всегда можно отделить от эмуляции.

Архитектура и техническая реализация deception-схем

Чтобы эффективно обходить защитные механизмы, Red Team должен досконально понимать, как они устроены. На практике «все порты открыты» реализуется несколькими техническими путями.

Portspooft и маршрутизация через iptables

Portspooft остается классическим инструментом для создания иллюзии 65 535 открытых TCP-портов. Инструмент не биндится на все порты в операционной системе. Вместо этого используется правило межсетевого экрана (например, iptables или nftables), которое перенаправляет все входящие TCP-запросы SYN на один скрытый локальный порт, который прослушивает демон Portspooft.

Практика: Конфигурация маршрутизации для Portspooft

На стороне сервера Blue Team настраивает перенаправление трафика со всех портов (кроме легитимного, например 22) на порт демона Portspooft (4444):

```
# Включение маршрутизации и перенаправление всех новых TCP-соединений
iptables -t nat -A PREROUTING -i eth0 -p tcp -m tcp --dport 1:65535 -j REDIRECT --to-ports 4444
# Исключение легитимного SSH порта (чтобы администраторы могли зайти)
iptables -t nat -I PREROUTING 1 -i eth0 -p tcp --dport 2222 -j ACCEPT
```

Когда сканер атакующего (например, Nmap) отправляет пакет SYN на любой порт, ядро Linux перехватывает его, редиректит на Portspooft, который послушно отвечает SYN-ACK. Сканер фиксирует порт как открытый.

Но Portspooft идет дальше: на каждый запрос, отправленный на установленное соединение (например, при запуске Nmap со скриптами детекции версий -sV), демон генерирует и возвращает валидный с точки зрения сигнатуры баннер из базы регулярных выражений (более 8000 сигнатур). Это ломает попытки сканера автоматически определить реальный сервис.

NAT/iptables-редиректы и самописные декои (Decoys)

Более примитивный вариант обходится без сложных баз сигнатур. Защитники могут написать простой Python или Go-демон, который слушает сокет и при любом подключении отдает псевдослучайный мусорный баннер или бесконечно удерживает соединение открытым, отправляя по 1 байту в секунду.

```
# Простейший tarpit-сервер на Python для удержания соединений
import socket
import time
def start_tarpit(port):
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.bind(('0.0.0.0', port))
```

```
s.listen(100)
print(f"Tarpit слушает на {port}...")

while True:
    conn, addr = s.accept()
    print(f"Соединение от: {addr}")
    try:
        # Отправка фальшивого баннера с задержкой
        conn.send(b"220 ESMTP Postfix\r\n")
        while True:
            time.sleep(10)
            conn.send(b" ") # Поддержание сессии живой (Keep-Alive)
    except Exception:
        conn.close()
```

Ошибка реализации самописных ловушек

Такие декои предельно легко детектируются. Они отдают статичные баннеры, не умеют обрабатывать реальные запросы (например, HTTP GET) и не имеют полноценной stateful-логики. Ответ на любой ввод либо одинаковый, либо сервер просто закрывает соединение.

Коммерческие Honeypot-платформы и Deception-системы

Энтерпрайз-сети (особенно в 2026 году) шагнули дальше скриптов энтузиастов. Коммерческие платформы (аналоги Portspooft Pro, системы от Illusive Networks или TrapX) разворачивают целые «тёмные подсети» (Darknets / Tarpits). Они не просто эмулируют 65k портов на одном IP, они эмулируют /24 подсети виртуальных машин с маршрутизаторами, Windows-доменами и SCADA-системами. Сканирование такого сегмента занимает у атакующего недели, генерируя алерты в SIEM с первого же SYN-пакета.

Взгляд со стороны атакующего (Red Team)

Для нас, как атакующих, встреча с подобной системой выглядит как резкое изменение поведения стандартного инструментария.

Семантика L3/L4: Сетевой уровень и Nmap / Masscan

Сетевые сканеры работают на основе семантики TCP-флагов. В базовой конфигурации они слепы к прикладному контексту.

- **Nmap SYN-скан (-sS):** Отправляет SYN, получает SYN-ACK на все 65k портов. Nmap рисует картину: 65 535 open ports.
- **Nmap Connect-скан (-sT):** Успешно проходит полный 3-way handshake на каждый порт.

- **Masscan / ZMap:** Асинхронные сканеры просто захлебываются. При скорости 10 000 пакетов в секунду, сканирование одного deception-хоста выдаст 65 тысяч ответов. Masscan зафиксирует "все открыто", не пытаясь анализировать причины.

```
# Типичный вывод Nmap при сканировании Portspooof
$ nmap -sV -p 1-100 192.168.1.50
Starting Nmap 7.93 ( https://nmap.org )
Nmap scan report for 192.168.1.50
Host is up (0.0021s latency).
PORT      STATE SERVICE VERSION
1/tcp     open  tcpmux?
2/tcp     open  ssh      OpenSSH 7.4 (protocol 2.0)
3/tcp     open  http     Apache httpd 2.4.6 ((CentOS))
4/tcp     open  smtp     Postfix smtpd
...
100/tcp   open  newacct?
100 ports scanned. All ports are open.
Service detection performed. Please report any incorrect results...
```

При использовании `nmap -sV` (определение версий), Nmap начинает слать зондирующие пакеты (probes) из файла `nmap-service-probes`. Ловушка отвечает валидными строками. В итоге скан 100 портов, который должен занять 2 секунды, занимает минуты, а полный скан всех портов — десятки часов.

Провал прикладного уровня (L7) и Service Detection

Основная слабость децептивных систем кроется в невозможности дешево и достоверно имитировать **поведенческий анализ протокола (State Machine)**. Сервис может представиться как SSH, отправив `SSH-2.0-OpenSSH_8.2p1 Ubuntu-4ubuntu0.1`, но он не знает, как вычислить криптографические ключи и ответить на пакет алгоритмов обмена (KEXINIT).

Если мы берем HTTP, ловушка может отдать заголовок `Server: nginx`, но при отправке нестандартного HTTP-глагола (например, `PROPFIND`) или запроса с chunked-кодированием, эмулятор либо отдает тот же дефолтный 200 OK с пустым телом, либо разрывает TCP-сессию. Это мгновенно выдает его искусственную природу.

Преимущества подхода «Все порты открыты» для Blue Team

Почему SOC-инженеры любят этот метод, несмотря на его очевидные недостатки для сетевых инженеров?

1. **Радикальное усложнение и удорожание разведки:** Полный порт-скан с определением версий и скриптами (`nmap -A -p-`) по одному хосту может растянуться на 10–12 часов вместо пары минут. Автоматизированные ботнеты и AI-разведчики просто упираются в таймауты и сбрасывают цель.

2. **Убийство автоматизированного Vulnerability Management:** Инструменты вроде Nessus, OpenVAS или сканеры уязвимостей периметра сойдут с ума. Они попытаются применить плагины к тысячам ложных сервисов, сгенерировав отчет на сотни тысяч страниц с 99.9% ложных срабатываний (False Positives). Найти в этом отчете реальную CVE невозможно.
3. **Детект и атрибуция атакующих TTPs:** Защитники видят каждое движение. Они собирают чистейший профиль: порядок сканирования портов, используемые payload-сигнатуры, тайминги, реакции сканера на нестандартные TCP-опции. Deception выступает как идеальный сенсор.
4. **Соккрытие реальной поверхности атаки:** Где-то среди 65 535 портов висит уязвимый корпоративный RDP. Найти его «вслепую» — как искать иголку в стоге сена, который сам генерирует фальшивые иголки.

Важно: Синергия с SIEM

Эффективность подхода работает на 100%, только если логи с honeypot-маршрутизатора напрямую пишутся в SIEM с автоматизированным реагированием (SOAR). Попадание IP-адреса атакующего в tarpit должно триггерить блокировку его адреса на периметральном WAF или Firewall в течение секунд, лишая его возможности найти настоящие сервисы.

Уязвимости метода и риски (Красные флаги для Red Team)

Метод «все порты открыты» — это не броня. Это маскировочная сеть. И как у любой маскировки, у неё есть демаскирующие признаки (OPSEC failures), которые мы активно эксплуатируем.

Риски внедрения для бизнеса (Self-DoS)

Удержание десятков тысяч соединений от асинхронных сканеров массово потребляет ресурсы таблицы состояний брандмауэра (conntrack table). Агрессивный скан от распределенного ботнета (или Red Team с использованием нескольких нод Masscan) может привести к переполнению таблицы conntrack (nf_conntrack: table full, dropping packet), что вызовет Отказ в Обслуживании (DoS) для легитимных сервисов на этом же IP.

Инконсистентные баннеры (Шизофрения сервисов)

От порта к порту эмулятор выдает безумные сочетания. На 80 порту может отвечать LDAP, на 22 порту — Oracle DB, а на 443 — FTP. Подобное смешение технологических стеков не встречается ни в одной реальной Enterprise-среде. Более того, при повторном сканировании одного и того же порта через 5 минут эмулятор может выдать совершенно другой сервис. Это мгновенный сигнал для атакующего: *«Мы в deception-зоне»*.

Буквальное применение метода как Анти-Паттерн

Если администратор решает не эмулировать, а *реально* поднять сервисы-пустышки на тысячах портов (например, сотни контейнеров Docker с разными демонами), это катастрофа. Каждая реальная точка входа — это потенциальная 0-day уязвимость. Патч-менеджмент 65 000 сервисов невозможен физически. Поэтому базовое правило кибергигиены: **минимизация открытых портов** остается абсолютной истиной.

Методология обхода ловушек: Инструментарий Red Team

Поняв, что перед нами honeypot или Portspooft, мы кардинально меняем подход. Широкоформатное сканирование отменяется.

1. Пассивный OSINT и исторические данные

Нам часто не нужно сканировать хост в реальном времени. В 2026 году базы данных телеметрии интернета хранят историю изменений инфраструктуры. Мы обращаемся к таким инструментам, как Shodan, Censys, Hunter.how или FOFA.

Мы смотрим исторические срезы IP-адреса *до того*, как Blue Team установил Portspooft. Обычно легитимные сервисы (22, 443, 3389) остаются на тех же портах, просто теперь они окружены шумом. Мы берем этот узкий список и работаем только с ним.

2. Валидация L7 и фаззинг State Machine (Контр-Децепция)

Мы перестаем верить баннерам и TCP-флагам. Мы натравливаем протокол-aware инструменты (например, decoy-hunter). Они устанавливают соединение и иницируют реальный диалог по стандартам RFC.

Практика: Скрипт обхода ловушки для протокола SSH

Этот пример на Python показывает, как Red Team отличает настоящий SSH-сервер от эмулятора Portspooft. Скрипт не просто читает баннер, он пытается получить от сервера KEX-пакет алгоритмов, который Portspooft сгенерировать не в состоянии.

```
import socket
import struct
def is_real_ssh(ip, port):
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.settimeout(3.0)
        s.connect((ip, port))

        # 1. Читаем баннер (Portspooft отдаст его)
        banner = s.recv(1024)
        if b"SSH" not in banner:
```

```

        return False

# 2. Отправляем наш клиентский баннер
s.send(b"SSH-2.0-RedTeam_Client\r\n")

# 3. Ждем пакет KEXINIT от сервера.
# Настоящий SSH сервер ответит бинарным пакетом обмена ключами.
# Portspooof либо промолчит, либо пришлет текстовый мусор.
data = s.recv(1024)

# Проверяем структуру бинарного SSH-пакета (длина пакета + padding)
if len(data) > 5:
    packet_len = struct.unpack('!I', data[0:4])[0]
    # В KEXINIT первый байт payload обычно равен 20 (0x14)
    if data[5] == 20:
        return True
    return False

except Exception:
    return False
finally:
    s.close()

# Использование
if is_real_ssh("192.168.1.50", 2222):
    print("Найден РЕАЛЬНЫЙ SSH сервис!")
else:
    print("Эмулятор/Honeypot обнаружен.")

```

Используя подобные скрипты-адаптеры для HTTP, RDP, SMTP и баз данных, Red Team отфильтровывает все 65 000 фейковых портов за считанные минуты, оставляя 2–3 настоящих порта для дальнейшей эксплуатации.

Сравнительный анализ стратегий защиты периметра

Мера защиты	Суть подхода	Преимущества (Плюсы)	Ограничения (Минусы)
Минимизация портов (Default Deny)	Оставить только необходимые сервисы, строго фильтровать остальное через ACL.	Минимальная поверхность атаки. Предсказуемое поведение инфраструктуры. Простота сопровождения и патч-менеджмента.	Не дает активной децепции. Атакующий быстро картографирует периметр и переходит к эксплуатации.
Port Knocking / ZTNA (Zero Trust)	Порты закрыты извне. Доступ открывается только после криптографической валидации или последовательности стуков.	Делает поверхность нулевой (Invisible OOB). Сильно усложняет жизнь и сканерам, и АРТ-группировкам.	Сложность внедрения и поддержки. При сбое архитектуры легитимные пользователи теряют доступ к бизнесу.
Tarpits (Лабреа-подобные)	Удерживать TCP-соединения сканеров открытыми максимально долго, отправляя TCP Window = 0.	Истощает ресурсы сканеров. Сводит скорость Nmap к нулю. Генерирует массивную телеметрию.	Огромный риск Self-DoS. Исчерпание сокетов на сетевом оборудовании. Необходимость тонкого тюнинга.
«Все порты открыты» (Portspoof)	Сделать каждый порт видимо `open` и эмулировать фальшивые сервисы.	Радикальное усложнение автоматического сканирования. Мощный слой Active Defense. Отличный детект разведки.	OPSEC провал (высокая детектируемость). Легко обходится протокол-aware тулзами (L7 валидация). Узкая применимость (только DMZ/Darknets).

Рекомендации по внедрению Deception-технологий (Для Blue Team / Архитекторов)

- **Не внедряйте в продуктивных подсетях:** Развешивание тысяч ложных сервисов внутри корпоративного LAN приведет к коллапсу собственных систем мониторинга ИТ-инфраструктуры.
- **Используйте "Darknets":** Выделяйте целые неиспользуемые подсети на внешнем периметре исключительно под Deception. Любой пакет, направленный туда — это 100% инцидент, не требующий триажа.
- **Интегрируйте с автоматикой:** Deception без реагирования — пустая трата ресурсов. honeypot должен автоматически обновлять правила брандмауэра для изоляции сканирующего IP.
- **Скрывайте реальные сервисы за ZTNA:** Если вы используете Portspooft, убедитесь, что единственные "реальные" доступы спрятаны за VPN или Zero Trust Network Access. Настоящий SSH вообще не должен светиться в интернете.

Вывод (Вердикт Red Team)

Подход «все порты открыты» — это великолепный инструмент для фильтрации "интернет-шума", ботнетов и начинающих исследователей (script kiddies). Он ломает примитивный Kill Chain и заставляет дешевые автоматизированные сканеры захлебываться в мусоре.

Однако, как чистая защитная мера против целенаправленной (Targeted) атаки или АРТ-группировок, он не заменяет базовую кибергигиену. Для опытного Red Team столкновение с deception-периметром — это не стена, а просто смена парадигмы. Мы убираем «широкий бредень» массированного сканирования сетевого уровня, переходим к пассивному OSINT, анализу исторических данных и применению хирургически точных L7-фаззеров. В 2026 году этот метод должен быть лишь одним из слоев (Defense-in-Depth), интегрированным с надежной архитектурой Zero Trust.

Часто задаваемые вопросы (FAQ)

1. Может ли Nmap распознать ловушки типа Portspooft автоматически?

Сам по себе Nmap, без специальных NSE-скриптов, обманывается легко. Ключ -sV полагается на регулярные выражения, которые Portspooft успешно подделывает. Однако существуют специализированные скрипты и надстройки, которые анализируют консистентность ответов, что позволяет Nmap заподозрить honeypot.

2. Защищает ли метод «все порты открыты» от DDoS атак?

Категорически нет. Наоборот, он может усугубить ситуацию. Отвечая на каждый SYN-пакет созданием сессии (SYN-ACK) и выделяя ресурсы для эмуляции баннера, сервер сам становится жертвой исчерпания ресурсов (State Table Exhaustion).

3. Как избежать Self-DoS при использовании Portspooft?

Необходимо настраивать строгие лимиты в iptables (модуль limit или hashlimit) для входящих SYN-пакетов, а также увеличивать параметры ``nf_conntrack_max`` в ядре Linux. Важно использовать TCP SYN Cookies (`net.ipv4.tcp_syncookies = 1`).

4. Можно ли спрятать реальный веб-сервер (80/443) среди фейковых портов на одном IP?

Можно с помощью правил маршрутизации (например, исключив порты 80 и 443 из правила REDIRECT). Но для Red Team не составит труда найти их, отправив валидный HTTP GET запрос по всем открытым портам и отфильтровав те, что вернули реальный контент сайта, а не мусорный ответ.

5. Что такое TTPs в контексте разведки?

TTPs (Tactics, Techniques, and Procedures) — это шаблоны поведения атакующего. В данном случае это то, какие инструменты используются (Nmap, Masscan), в каком порядке сканируются порты (линейно или случайно), и как атакующий реагирует на задержки. Deception-системы отлично профилируют эти паттерны.

6. Насколько актуален Portspooft в 2026 году?

Оригинальный открытый код Portspooft стареет, но концепция получила мощное развитие. Сейчас этот функционал встроен в продвинутые Enterprise-решения для защиты конечных точек и сетевого периметра (Deception Platforms). Принципы работы остались теми же, но увеличилась реалистичность эмуляции протоколов.

7. Заменяет ли Deception традиционный Firewall?

Нет. Deception — это надстройка для обнаружения и задержки. Firewall (или NGFW) должен по-прежнему блокировать нелегитимный трафик к внутренним сегментам. Ловушки ставятся параллельно или перед брандмауэром в специально изолированных зонах (DMZ).

8. Какой самый надежный способ скрыть сервисы от сканирования?

Наиболее криптографически надежный подход на сегодня — это Single Packet Authorization (SPA) / Port Knocking или решения класса ZTNA, где порт полностью закрыт на сетевом уровне (DROP/REJECT) до тех пор, пока клиент не отправит криптографически подписанный пакет авторизации.